



Experiment 7

Student Name: Analava Bera
Branch: CSE
Semester: 6th
Subject Name: System Design

UID: 23BCS13611
Section/Group: KRG 3-A
Date of Performance: 02/04/2026
Subject Code: 23CSH-314

1. Aim: To design and develop a ride-sharing system that enables users to book rides and drivers to accept requests with real-time tracking and efficient matching.

2. Objective:

- Provide on-demand ride booking
- Enable real-time driver allocation
- Ensure fast and reliable matching
- Support secure payment system
- Provide live tracking & navigation

3. System Requirements:

A. Functional Requirements

- User(Rider)
 - a. Get fare estimation
 - b. Request ride
 - c. Choose ride type(car, bike, etc.)
 - d. Track ride in real-time
 - e. Make payments
 - f. Rate driver
- Driver
 - a. Accept/reject ride
 - b. Navigate to pickup & drop
 - c. Update location continuously
 - d. Complete ride
- Core System
 - a. Match rider with nearby driver
 - b. Real-time tracking
 - c. Ride history
 - d. Notifications to drivers
 - e. Users should be able to search videos based on title, creator name, or tags.

B. Non-Functional Requirements

- Scalability
Target: ~1.5million users & drivers
- Performance
- Driver assignment within < 1 minute

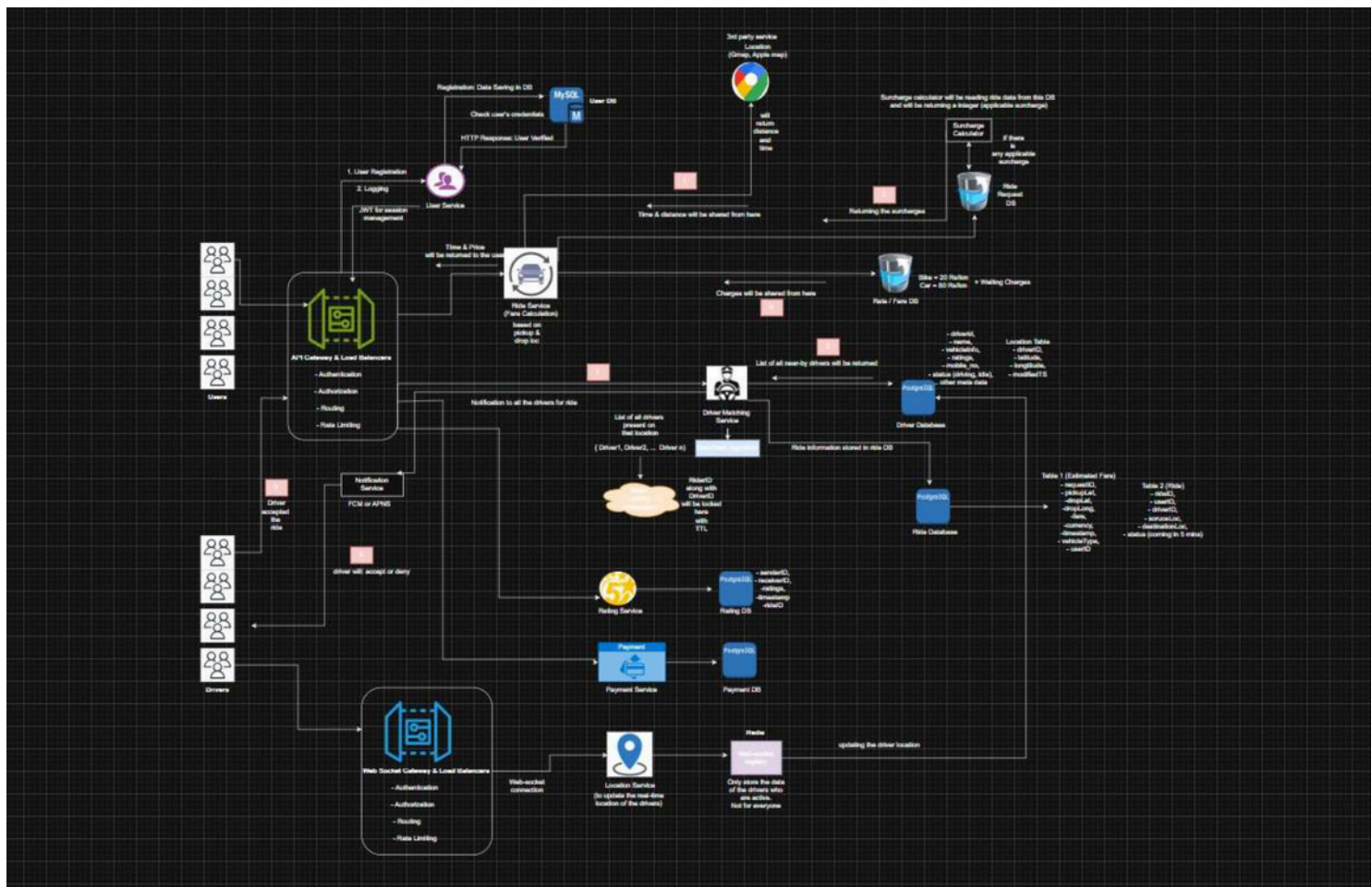
CAP Theorem

Strong consistency → driver not assigned to multiple users

Eventual consistency → for user data / updates Availability

- Latency:
High availability for users (system always accessible)
Video playback latency should be low (around 50–80 ms) with minimal buffering during streaming

4. Low Level Design (LLD):





5. Scalability Solution

- Use Microservices Architecture (separate services for user, driver, ride, payment)
- Implement Load Balancers to distribute traffic
- Apply Database Sharding (split data by region/location)
- Use Database Replication for high availability
- Implement Caching (Redis) for faster access to frequent data
- Use Geo-Spatial Indexing for efficient nearby driver search.

6. Learning Outcomes (What I Have Learnt)

- Understand system design of real-world apps (Ola/Uber)
- Learn functional vs non-functional requirements
- Gain knowledge of scalable architecture design
- Understand CAP theorem (Consistency vs Availability)
- Learn real-time data handling (location tracking).